Модуль 3.1: AI-агенты — Теория и инструментарий

Формат: Митап + self-paced reading

Время: ~2 часа (митап) + доп. материалы

Пререквизиты: Модуль 1 (Model Literacy), Модуль 2 (LLM System Design)

- Цели модуля
- 1. Что такое AI-агент?
  - Наглядный пример
- 2. Формула агента
  - LLM — мозг агента
  - State — память агента
    - Tools — руки агента
    - Policy — правила поведения
    - Feedback Loop — цикл обратной связи
  - 3. Где и почему агенты ломаются
    - 3.1 Compounding Errors — накопление ошибок
    - 3.2 Hallucinated Actions — выдуманные действия
    - 3.3 Context Window Overflow — переполнение контекста
    - 3.4 Infinite Loops — бесконечные циклы
    - 3.5 Misaligned Goals — несогласованные цели
    - 3.6 Tool Misuse — неправильное использование инструментов
  - 4. Автономность: почему меньше = лучше
    - Спектр автономности
    - Почему полная автономность опасна
    - Правило 80/20 для агентов
    - Когда автономность оправдана
  - 5. Human-in-the-Loop (HITL)
    - Что такое HITL
    - Когда HITL обязателен
    - Паттерны HITL
    - Баланс: HITL vs Automation
  - 6. Инструментарий для создания агентов
    - 6.1 Claude Code / Claude Agents
    - 6.2 LangChain / LangGraph
    - 6.3 n8n
    - 6.4 deBridge ai-dev-kit
    - Сравнение инструментов
  - 7. Границы ответственности агента
    - Что агент должен делать
    - Что агент НЕ должен делать
    - Принцип минимальных привилегий
    - Audit и Observability
  - Демо и упражнения
    - Demo 1: Анатомия агента
    - Demo 2: HITL в действии
    - Упражнение
  - Ключевые выводы
  - Дополнительные материалы

Цели модуля

После этого модуля вы сможете:

- Понимать что такое AI-агент и чем он отличается от простого LLM-вызова
- Знать формулу агента и её компоненты
- Понимать где и почему агенты ломаются

- Принимать решение: автономный агент vs детерминированный workflow
- Выбирать инструмент для создания агентов под задачу

---

1. Что такое AI-агент?

Часто возникает путаница: где заканчивается простой вызов LLM и начинается агент? Разница фундаментальна. **Простой LLM-вызов** — это калькулятор. Ввод → Вывод → Конец.. **AI-агент** — это сотрудник. Ему ставят задачу, а он сам планирует шаги, ошибается, исправляется и идет к цели цикличными итерациями.

Если классический скрипт — это жесткая логика `if-then`, то агент — это цикл `Plan → Act → Observe → Reflect`.

Простой LLM-вызов — это одиночный запрос-ответ:

```
User: "Переведи текст на английский"
LLM: "Here is the translation..."
```

Это как калькулятор: дали задачу → получили ответ → конец.

AI-агент — это система, которая может:

- Планировать последовательность действий
- Выполнять действия (вызывать инструменты)
- Наблюдать за результатами
- Адаптировать план на основе наблюдений
- Работать в цикле до достижения цели

Возращаясь к аналогии с сотрудником, действует следующий алгоритм: получил задачу → разбил на шаги → выполняет → сталкивается с проблемой → адаптируется → продолжает → докладывает результат.

Наглядный пример

Простой LLM-вызов:

```
User: "Какая погода в Москве?"
LLM: "Я не знаю текущую погоду, так как не имею доступа к интернету."
```

Агент:

```
User: "Какая погода в Москве?"

Agent thinks: Мне нужно узнать текущую погоду. У меня есть инструмент we
Agent action: weather_api.get_weather("Moscow")
Observation: {"temp": 15, "condition": "cloudy", "humidity": 70}
Agent thinks: Получил данные, могу ответить пользователю.
Agent response: "В Москве сейчас 15°C, облачно, влажность 70%."
```

Ключевое отличие: агент может **действовать** во внешнем мире и **адаптироваться** к результатам.

---

2. Формула агента

Любой агент, от простейшего бота до сложного ассистента, декомпозируется на 5 компонентов. Если убрать один из них — система развалится.

```
Agent = LLM + State + Tools + Policy + Feedback Loop
```

LLM — мозг агента

LLM выполняет функцию Это **Reasoning Engine**. Модель здесь не для того, чтобы хранить знания (база данных справится лучше), а для того, чтобы принимать решения и планировать.

Важно понимать: LLM не "думает" в человеческом смысле. Он предсказывает следующий токен. Но при правильном промпте это предсказание может выглядеть как планирование и рассуждение.

Выбор LLM для агента:

- **Сложные задачи** → *GPT-5/Opus:* Для сложной оркестрации.
- **Простые задачи** → GPT-4o-mini, Claude Haiku, Gemini flash (быстрее, дешевле)
- **Приватность** → LLaMA, Mistral (локально)

State — память агента

Критически важный и часто игнорируемый компонент. LLM stateless по своей природе. **State** — это управляемое хранилище контекста. В коде это выглядит как типизированный объект.

State хранит:

- **Историю разговора** — что уже обсуждали
- **Промежуточные результаты** — что уже сделали
- **Контекст задачи** — что пытаемся достичь
- **Scratchpad** — заметки агента для себя

Без state агент не помнит, что делал шаг назад. Каждый шаг был бы "с чистого листа".

Важно: State — это НЕ долгосрочная память модели. Это явно управляемое хранилище в вашем коде.

```typescript
// Пример state в коде (TypeScript)
interface AgentState {
  goal: string;
  currentStep: number;
  completedSteps: string[];
  pendingSteps: string[];
  observations: string[];
}

const state: AgentState = {
  goal: "Создать PR с фиксом бага",
  currentStep: 3,
  completedSteps: ["найден файл", "проанализирован баг"],
  pendingSteps: ["написать фикс", "запустить тесты", "создать PR"],
  observations: ["Баг в строке 42: неправильная проверка null"]
};
```

Tools — руки агента

Это руки агента. Функции, связывающие вероятностный мозг с детерминированным миром. Принцип прост: Агент генерирует *JSON-описание* намерения (`{ tool: "search", query: "BTC" }`), а рантайм выполняет реальный код .

Примеры tools:

- `search(query)` — поиск в интернете
- `read_file(path)` — чтение файла
- `write_file(path, content)` — запись файла
- `run_command(cmd)` — выполнение команды
- `create_jira_ticket(...)` — создание задачи
- `send_slack_message(...)` — отправка сообщения

Принцип: агент описывает ЧТО хочет сделать, система выполняет КАК.

```
// Агент генерирует вызов (structured output):
const toolCall = { tool: "search", query: "BTC price USD" };

// Система выполняет и возвращает результат:
const result = { result: "Bitcoin is trading at $67,432" };
```

Важно: Качество описания tools критически влияет на качество агента. Плохое описание → агент не понимает когда и как использовать tool.

Policy — правила поведения

Это ваши `Guardrails`. Список того, что агенту **запрещено** делать.

- **Что агент может делать** — whitelist действий
- **Что агент НЕ может делать** — blacklist, ограничения
- **Когда эскалировать** — при каких условиях спрашивать человека
- **Как приоритизировать** — что важнее

Примеры policy rules:

```
- Никогда не удаляй файлы без подтверждения пользователя
- При ошибке API — попробуй 3 раза, потом эскалируй
- Не трать более $1 на один запрос к LLM
- Если confidence < 80% — спроси уточнение
- Критические действия (деплой, транзакции) — всегда с HITL
```

Policy — это ваш способ контролировать автономность агента.

Feedback Loop — цикл обратной связи

Feedback Loop — это механизм, который позволяет агенту учиться на результатах своих действий в рамках одной сессии.

```
Plan → Act → Observe → Reflect → Re-plan → Act → ...
```

Observe: Агент получает результат действия (успех/ошибка, данные).

Reflect: Агент анализирует: "Приблизило ли это меня к цели? Что пошло не так? Что делать дальше?"

Re-plan: Агент корректирует план на основе наблюдений.

Без feedback loop агент выполнял бы план вслепую, игнорируя результаты.

---

3. Где и почему агенты ломаются

Агенты — мощный инструмент, но также и хрупкая конструкция, и они ломаются чаще, чем хотелось бы. Понимание точек отказа критически важно для проектирования надёжных систем.

3.1 Compounding Errors — накопление ошибок

Каждый шаг агента имеет вероятность ошибки. При цепочке шагов ошибки накапливаются.

```
Если каждый шаг имеет 95% успеха:
- 1 шаг: 95%
- 3 шага: 95%³ = 86%
- 5 шагов: 95%⁵ = 77%
- 10 шагов: 95%¹⁰ = 60%
```

Вывод: Длинные цепочки автономных действий — это риск. Чем больше шагов, тем ниже надёжность.

Mitigation:

- Минимизировать количество шагов
- Добавлять checkpoints и валидацию

- Использовать Human-in-the-Loop на критических шагах

3.2 Hallucinated Actions — выдуманные действия

LLM может "выдумать" tool, который не существует, или вызвать tool с неправильными параметрами.

```
Agent: Вызываю deploy_to_production("v2.0")
System: Ошибка! Tool deploy_to_production не существует.
Agent: Ок, тогда вызываю production_deploy("v2.0")
System: Ошибка! Tool production_deploy не существует.
... бесконечный loop
```

Mitigation:

- Чёткое описание доступных tools в system prompt
- Валидация вызовов перед выполнением
- Few-shot примеры правильных вызовов
- Ограничение retry attempts

3.3 Context Window Overflow — переполнение контекста

При длинных сессиях state + history + observations могут превысить context window.

```
Шаг 1: 500 токенов (system + tools + history)
Шаг 10: 5000 токенов
Шаг 50: 25000 токенов → превышен лимит!
```

Mitigation:

- Summarization промежуточных результатов
- Sliding window для истории
- Selective memory — хранить только важное
- Явное управление state size

3.4 Infinite Loops — бесконечные циклы

Агент может застрять в цикле, повторяя одни и те же действия.

```
Agent: Поиск "API documentation"
Observation: Не найдено
Agent: Поиск "API documentation" (опять!)
Observation: Не найдено
Agent: Поиск "API documentation" (и снова!)
...
```

Mitigation:

- Max iterations limit
- Детекция повторяющихся действий
- Обязательный progress check: "Приблизился ли я к цели?"
- Timeout на всю задачу

3.5 Misaligned Goals — несогласованные цели

Агент может "понять" задачу не так, как вы имели в виду, и оптимизировать не то.

```
User: "Сделай код быстрее"
Agent: Удаляет все проверки и валидации (быстрее, но небезопасно!)
```

Mitigation:

- Чёткие, однозначные инструкции
- Constraints в policy ("не удаляй проверки безопасности")
- Preview/Review перед выполнением критических действий
- Тесты и валидация результата

3.6 Tool Misuse — неправильное использование инструментов

Агент может использовать tool не по назначению или с неправильными параметрами.

```
Agent: delete_file("/etc/passwd")  # Опасно!
Agent: send_email(to="all@company.com", body="SPAM")  # Массовая рассылк
```

Mitigation:

- Sandboxing — ограничение прав tools
- Confirmation для опасных действий
- Parameter validation
- Audit logging всех вызовов

---

4. Автономность: почему меньше = лучше

Спектр автономности

```
Полностью          Детерминированный        Полностью
ручной             workflow с AI            автономный
процесс            компонентами             агент
   |___________________|___________________|

   Низкий риск         Баланс              Высокий риск
   Низкая скорость     ✓ Оптимум           Высокая скорость
```

Почему полная автономность опасна

1. **Непредсказуемость** — невозможно гарантировать поведение
2. **Необратимость** — некоторые действия нельзя отменить
3. **Accountability** — кто отвечает за ошибку агента?
4. **Debugging** — сложно понять почему агент сделал X
5. **Trust** — пользователи не доверяют "чёрному ящику"

Правило 80/20 для агентов

> **80% агентов должны быть детерминированными workflows с AI-компонентами, не полностью автономными системами.**

Что это значит:

Детерминированный workflow:

```
1. Получить PR → фиксированный шаг
2. Извлечь diff → фиксированный шаг
3. [AI] Проанализировать изменения → AI-компонент
4. [AI] Сгенерировать комментарии → AI-компонент
5. Отправить review → фиксированный шаг
```

Структура workflow фиксирована. AI используется только там, где нужен reasoning.

Полностью автономный агент:

```
1. Получить задачу "сделай код-ревью"
2. Агент сам решает что делать, в каком порядке, какие tools использоват
3. Агент сам определяет когда закончить
```

Для большинства production задач первый подход надёжнее.

Когда автономность оправдана

- **Exploration tasks** — исследование без чёткого плана
- **Creative tasks** — генерация контента
- **Low-stakes tasks** — ошибка не критична

- **Human-supervised** — человек наблюдает в реальном времени

---

5. Human-in-the-Loop (HITL)

Что такое HITL

Human-in-the-Loop — паттерн, при котором человек включён в цикл работы агента как точка принятия решений или валидации.

```
Agent работает → Критическая точка → Пауза → Человек проверяет →
→ Approve/Reject → Агент продолжает или корректирует
```

Когда HITL обязателен

1. **Необратимые действия**
   - Удаление данных
   - Отправка сообщений клиентам
   - Финансовые транзакции
   - Деплой в production
2. **Высокая цена ошибки**
   - Медицинские рекомендации
   - Юридические документы
   - Критическая инфраструктура
3. **Недостаточная уверенность**
   - Confidence < threshold
   - Неоднозначная задача
   - Первые запуски нового агента
4. **Compliance требования**
   - Audit trail
   - Regulatory requirements
   - Внутренние политики безопасности

Паттерны HITL

1. Approval Gate — точка подтверждения

```
Agent: Я хочу удалить файл config.yaml. Подтвердите.
Human: [Approve] / [Reject]
Agent: (действует или останавливается)
```

2. Review & Edit — проверка и редактирование

```
Agent: Вот draft email для отправки клиенту: "..."
Human: (читает, редактирует)
Human: [Send edited version]
```

3. Escalation — эскалация при проблемах

```
Agent: Не могу найти решение после 3 попыток. Эскалирую.
Human: (подключается, решает)
```

4. Periodic Checkpoint — периодическая проверка

```
Agent: Выполнено 5 из 10 шагов. Промежуточные результаты: ...
Human: [Continue] / [Adjust] / [Stop]
```

Баланс: HITL vs Automation

```
        Слишком мало HITL          Слишком много HITL
        ─────────────────          ──────────────────
        Риск ошибок                Медленно
        Нет контроля               Утомляет человека
```

```
        Сложно доверять            Теряется смысл автоматизации

                    Оптимум: HITL на критических точках
```

---

6. Инструментарий для создания агентов

6.1 Claude Code / Claude Agents

Claude Code — инструмент для работы с Claude в терминале с поддержкой агентных возможностей.

Ключевые компоненты:

Компонент	Описание
[CLAUDE.md]()	Файл конфигурации агента — роль, правила, контекст
Skills	Расширения возможностей через промпты
Commands	Пользовательские команды
Hooks	Автоматические действия на события
MCP	Model Context Protocol — стандарт для tools

Когда использовать:

- Быстрое прототипирование агентов
- Интеграция с IDE (VS Code, JetBrains)
- Работа с кодом и файлами
- Когда нужен готовый agent runtime

Пример [CLAUDE.md]():

```
# Role
Ты — code reviewer для deBridge. Твоя задача — находить баги и уязвимос

# Rules
- Всегда проверяй на reentrancy в Solidity
- Не предлагай изменения без объяснения почему
- При обнаружении критической уязвимости — сразу сообщи

# Context
Репозиторий: debridge-contracts
Язык: Solidity
Стандарты: OpenZeppelin
```

6.2 LangChain / LangGraph

LangChain — фреймворк для создания LLM-приложений с готовыми абстракциями.

LangGraph — расширение для создания stateful агентов с графовой структурой.

Языки: Полноценная поддержка **Python** и **TypeScript** (нативная!). TypeScript-версия доступна через пакеты `@langchain/core`, `@langchain/langgraph`, `@langchain/anthropic` и др.

Ключевые концепции:

- **Chains** — последовательности вызовов
- **Agents** — LLM + Tools + Reasoning loop
- **Memory** — управление state и историей

- **Callbacks** — hooks для логирования и мониторинга

Когда использовать:

- Production-ready решения
- Сложные multi-step pipelines
- Нужна гибкость и кастомизация
- **TypeScript-стек** (как в deBridge!) — полноценная нативная поддержка

Пример LangGraph агента (TypeScript):

```
import { StateGraph, END } from "@langchain/langgraph";
import { ChatAnthropic } from "@langchain/anthropic";
import { BaseMessage, HumanMessage } from "@langchain/core/messages";

// Определяем state
interface AgentState {
  messages: BaseMessage[];
  nextAction: string;
}

// Определяем nodes (шаги)
async function reasoningNode(state: AgentState): Promise<Partial<AgentS
  const model = new ChatAnthropic({ model: "claude-sonnet-4-20250514" }
  const response = await model.invoke(state.messages);
  return { messages: [...state.messages, response] };
}

async function actionNode(state: AgentState): Promise<Partial<AgentStat
  // Выполняем действие на основе последнего сообщения
  const lastMessage = state.messages[state.messages.length - 1];
  // ... tool execution logic
  return state;
}

function shouldContinue(state: AgentState): "continue" | "end" {
  // Логика определения нужно ли продолжать
  return state.nextAction === "done" ? "end" : "continue";
}

// Строим граф
const workflow = new StateGraph<AgentState>({
  channels: {
    messages: { value: (a, b) => [...a, ...b], default: () => [] },
    nextAction: { value: (_, b) => b, default: () => "" },
  },
});

workflow.addNode("reason", reasoningNode);
workflow.addNode("act", actionNode);

workflow.addEdge("reason", "act");
workflow.addConditionalEdges("act", shouldContinue, {
  continue: "reason",
  end: END,
});

workflow.setEntryPoint("reason");

const app = workflow.compile();
```

6.3 n8n

n8n — low-code платформа для автоматизации с AI-нодами.

Ключевые возможности:

- Visual workflow builder
- 400+ интеграций (Slack, Jira, GitHub, etc.)
- AI nodes (OpenAI, Anthropic)
- Self-hosted или cloud

Когда использовать:

- Автоматизация бизнес-процессов
- Интеграции между сервисами

- Не хотите писать много кода
- Нужен visual debugging

Пример use case:

```
Trigger: Новый PR в GitHub
  → AI Node: Проанализировать diff
  → Condition: Есть проблемы?
    → Да: Создать комментарий в PR
    → Нет: Отправить в Slack "PR looks good"
```

6.4 deBridge ai-dev-kit

ai-dev-kit — внутренний toolkit deBridge с примерами и шаблонами.

Репозиторий: github.com/debridge-finance/ai-dev-kit

Что внутри:

- [CLAUDE.md](#) примеры для разных контекстов
- Гайды по claude-code-sdk
- MCP server examples
- Hooks и commands
- ESLint конфиги для архитектурных правил
- Ready-to-use schematics

Когда использовать:

- Новый проект в deBridge
- Нужен референс "как у нас принято"
- Ищете готовые шаблоны

Сравнение инструментов

Инструмент	Языки	Скорость старта	Гибкость	Production-ready	Когда выбирать
Claude Code	Any	⚡ Быстро	Средняя	✓	Прототипы, работа с кодом
LangChain/LangGraph	**TS**, Python	Средне	Высокая	✓✓	Production, кастомные pipelines
n8n	Low-code	⚡ Быстро	Средняя	✓	Интеграции, автоматизация
ai-dev-kit	**TS**	⚡ Быстро	Высокая	✓✓	deBridge проекты

---

7. Границы ответственности агента

Что агент должен делать

- Автоматизировать рутину

- Ассистировать в принятии решений
- Ускорять workflows
- Обрабатывать информацию

Что агент НЕ должен делать

- Принимать критические решения без человека
- Иметь доступ к чему не нужно
- Работать без audit trail
- Заменять экспертизу там, где она критична

Принцип минимальных привилегий

Агент должен иметь только те права, которые необходимы для задачи.

```
❌ Плохо: Агент имеет admin права "на всякий случай"
✓ Хорошо: Агент может только читать логи и создавать tickets
```

Audit и Observability

Каждое действие агента должно быть:

- **Logged** — записано что сделал
- **Traceable** — можно понять почему
- **Reversible** — можно откатить (где возможно)

Демо и упражнения

Demo 1: Анатомия агента

Разберём real-world агента на компоненты: LLM, State, Tools, Policy, Feedback Loop.

Demo 2: HITL в действии

Покажем как реализовать approval gate и escalation.

Упражнение

Спроектируйте агента для задачи:

> "Агент, который мониторит Slack-каналы поддержки и автоматически создаёт Jira-тикеты для проблем, требующих внимания разработчиков."

Подумайте:

1. Какие компоненты формулы агента нужны?
2. Где агент может сломаться?
3. Где обязателен HITL?
4. Какой уровень автономности оптимален?
5. Какие tools нужны?
6. Какой инструмент выбрать для реализации?

Ключевые выводы

1. **Агент = LLM + State + Tools + Policy + Feedback Loop**
2. **Агенты ломаются** — compounding errors, loops, hallucinations
3. **Автономность опасна** — 80% агентов должны быть детерминированными

4. **HITL обязателен** для критических и необратимых действий
5. **Выбирайте инструмент** под задачу: Claude Code, LangChain (TS/Python), n8n, ai-dev-kit
6. **Минимальные привилегии** — агент должен иметь только нужные права

Дополнительные материалы

Теория:

- 📄 ReAct: Reasoning and Acting in LLMs
- 📄 Toolformer: LLMs Can Teach Themselves to Use Tools
- 📺 Building LLM Agents — Harrison Chase (LangChain)

Инструменты:

- 📖 Claude Code Documentation
- 📖 LangGraph TypeScript Documentation
- 📖 LangChain.js Documentation
- 📖 n8n AI Documentation
- 📖 deBridge ai-dev-kit

Следующий модуль: AI-агенты (воркшоп) — практика создания агентов